SECURITY PRINCIPLES FOR SECURE SOFTWARE DESIGN

Secure Web Development

WHAT ARE SECURITY PRINCIPLES?

Principles and practices applications should adhere to in order to remain secure

Should be adapted based on the needs of the current project

Applies not only to web applications, but to IT systems as a whole

WHAT ARE SECURITY PRINCIPLES?

Principles and practices applications should adhere to in order to remain secure

Should be adapted based on the needs of the current project

Applies not only to web applications, but to IT systems as a whole

WHAT ARE SECURITY PRINCIPLES?

Principles and practices applications should adhere to in order to remain secure

Should be adapted based on the needs of the current project

Applies not only to web applications, but to IT systems as a whole

WHAT ARE SECURITY PRINCIPLES?

Principles and practices applications should adhere to in order to remain secure

Should be adapted based on the needs of the current project

Applies not only to web applications, but to IT systems as a whole

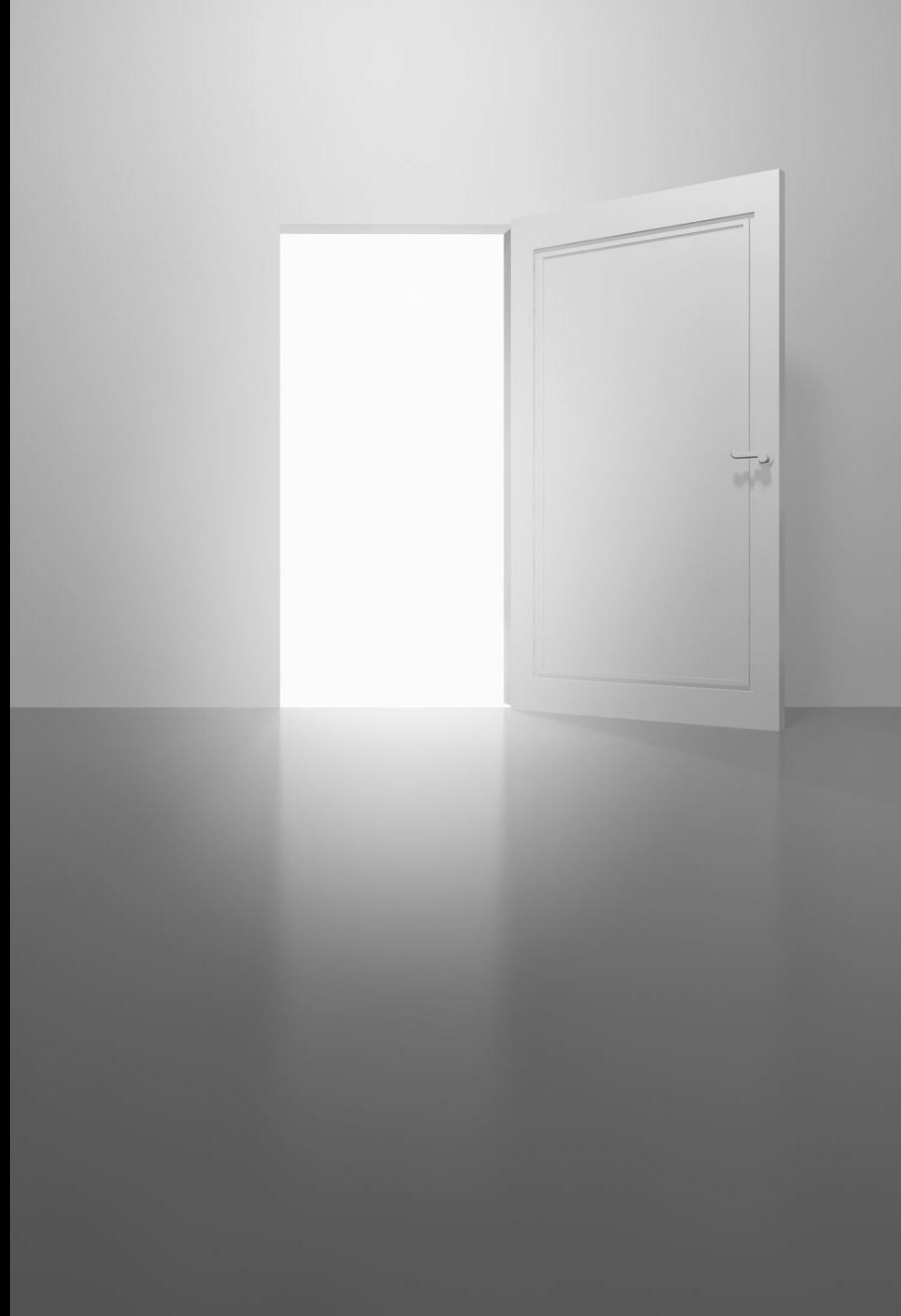
1. MINIMIZE ATTACK SURFACE AREA

Attack Surface Area – entry points in an application that may allow attackers to gain entry and eventually exploit vulnerabilities and compromise the application

The goal is to reduce the number of ways an attacker can exploit the application

Limiting entry points, removing unnecessary features, enforcing principle of least privilege, etc.

Practical Example: A company launches a mobile banking app. Instead of including a full social media feed or complex internal chat system that could have vulnerabilities, they only include core banking features like transfers and balance checks. By disabling unused ports and removing unnecessary third-party plugins from their web server, they further reduce the paths an attacker can take.



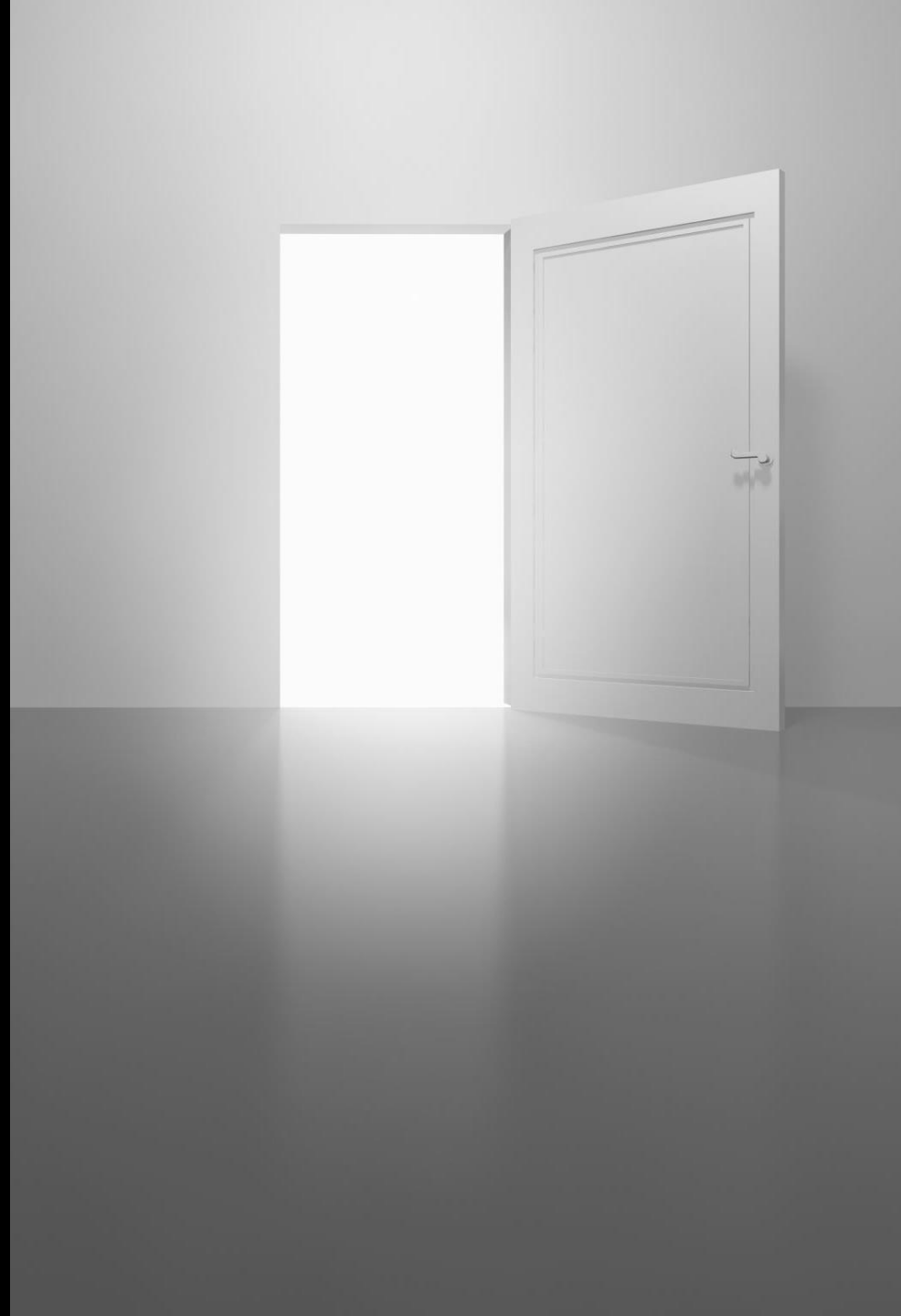
1. MINIMIZE ATTACK SURFACE AREA

Attack Surface Area – entry points in an application that may allow attackers to gain entry and eventually exploit vulnerabilities and compromise the application

The goal is to reduce the number of ways an attacker can exploit the application

Limiting entry points, removing unnecessary features, enforcing principle of least privilege, etc.

Practical Example: A company launches a mobile banking app. Instead of including a full social media feed or complex internal chat system that could have vulnerabilities, they only include core banking features like transfers and balance checks. By disabling unused ports and removing unnecessary third-party plugins from their web server, they further reduce the paths an attacker can take.



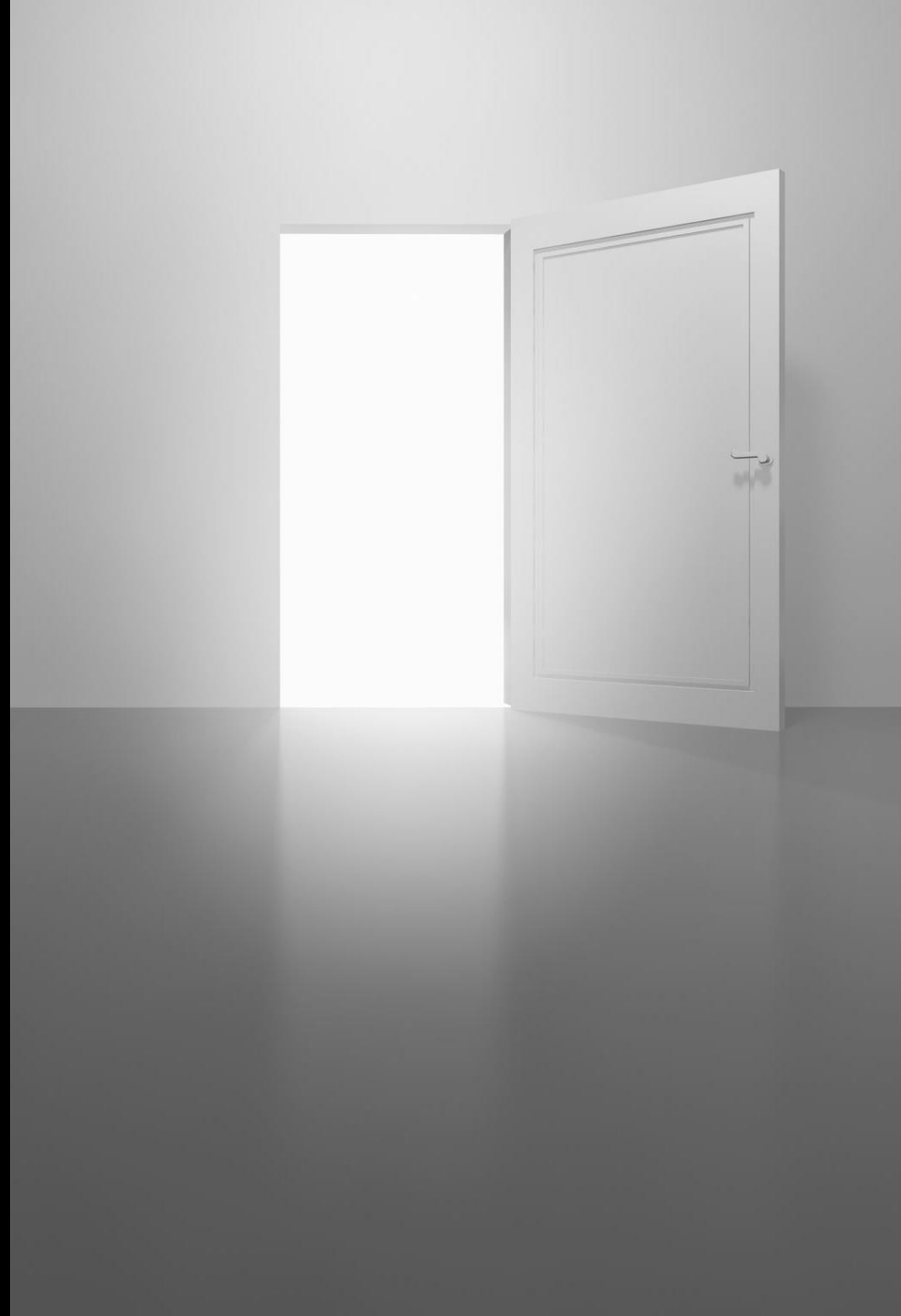
1. MINIMIZE ATTACK SURFACE AREA

Attack Surface Area – entry points in an application that may allow attackers to gain entry and eventually exploit vulnerabilities and compromise the application

The goal is to reduce the number of ways an attacker can exploit the application

Limiting entry points, removing unnecessary features, enforcing principle of least privilege, etc.

Practical Example: A company launches a mobile banking app. Instead of including a full social media feed or complex internal chat system that could have vulnerabilities, they only include core banking features like transfers and balance checks. By disabling unused ports and removing unnecessary third-party plugins from their web server, they further reduce the paths an attacker can take.



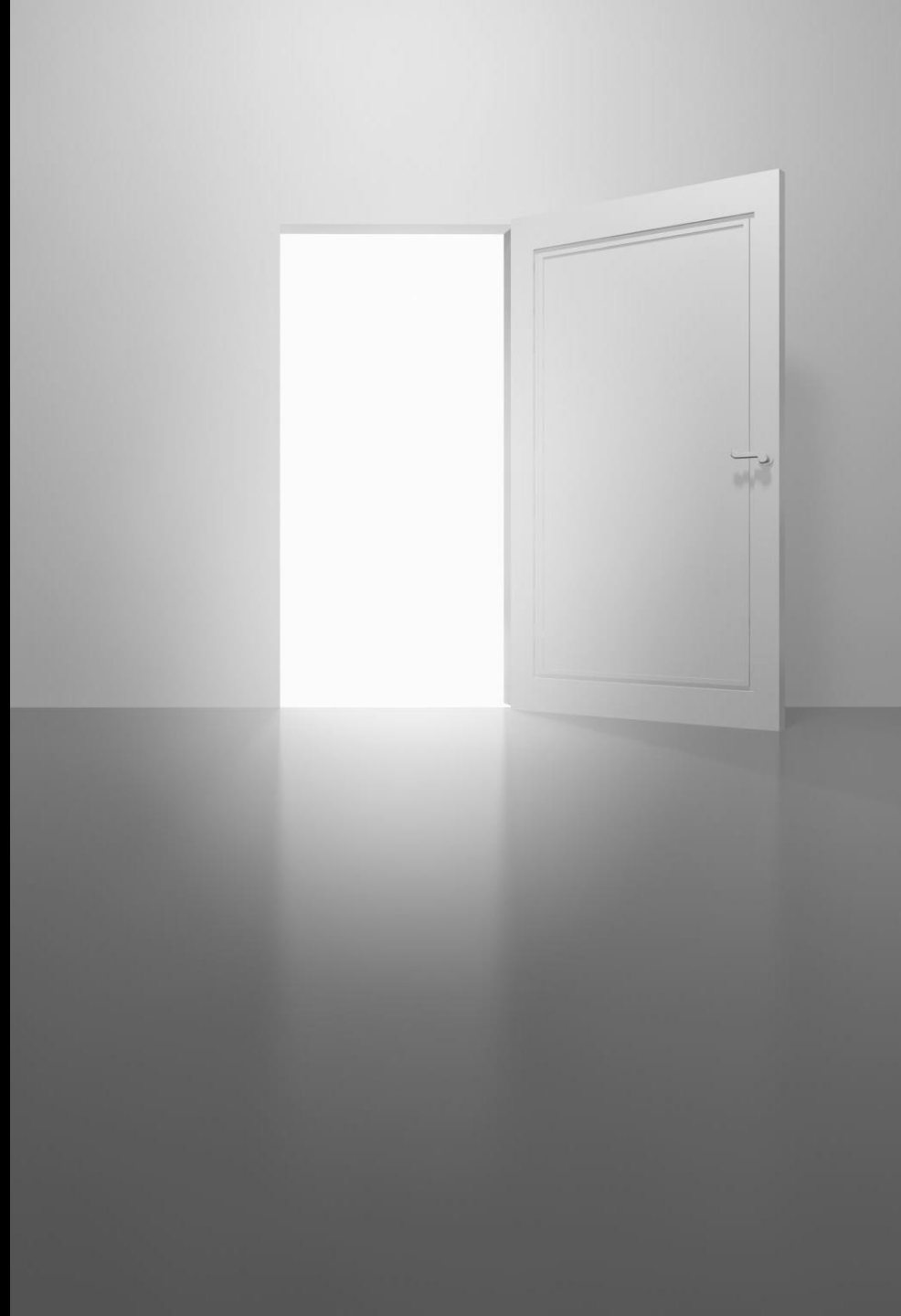
1. MINIMIZE ATTACK SURFACE AREA

Attack Surface Area – entry points in an application that may allow attackers to gain entry and eventually exploit vulnerabilities and compromise the application

The goal is to reduce the number of ways an attacker can exploit the application

Limiting entry points, removing unnecessary features, enforcing principle of least privilege, etc.

Practical Example: A company launches a mobile banking app. Instead of including a full social media feed or complex internal chat system that could have vulnerabilities, they only include core banking features like transfers and balance checks. By disabling unused ports and removing unnecessary third-party plugins from their web server, they further reduce the paths an attacker can take.



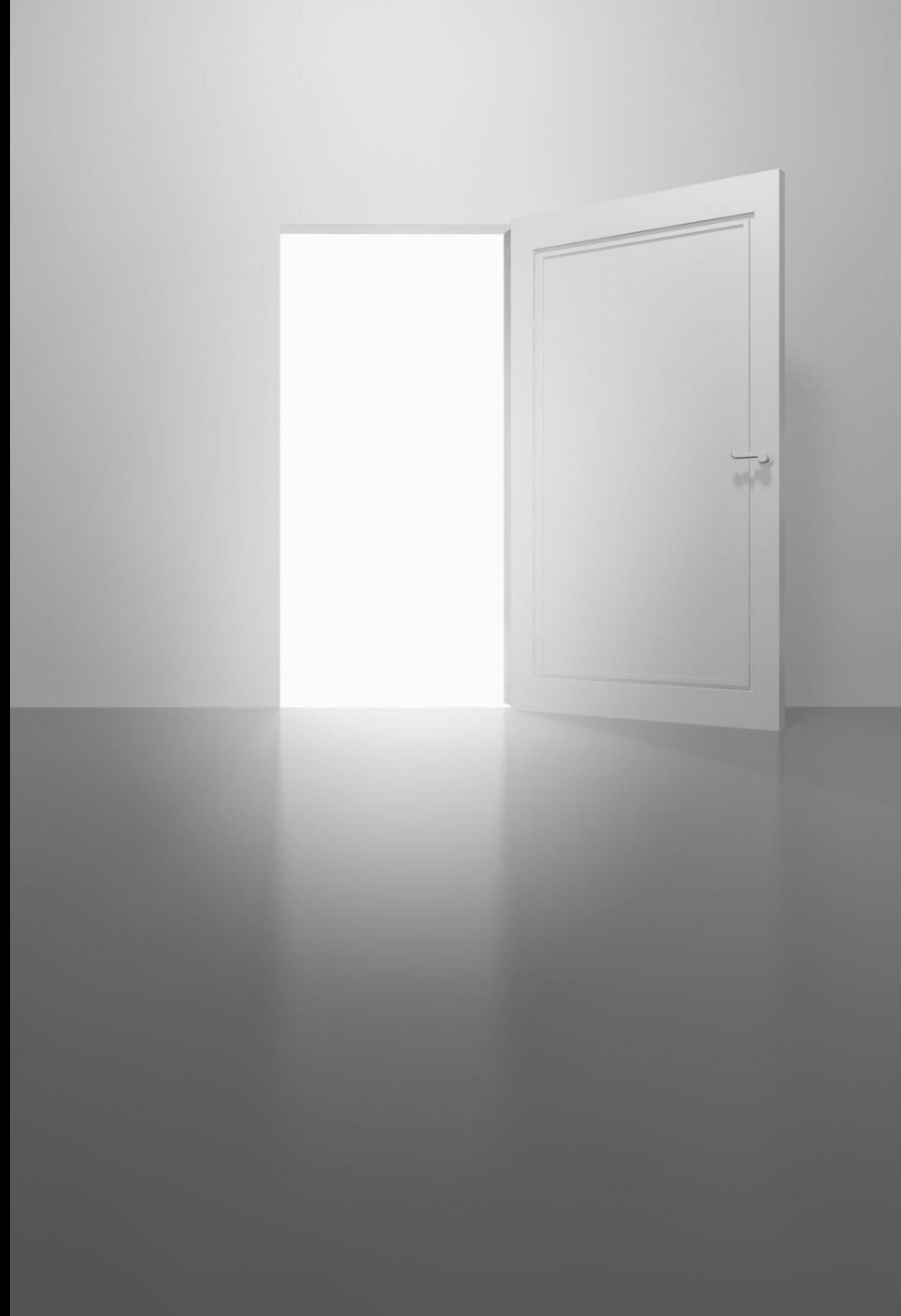
1. MINIMIZE ATTACK SURFACE AREA

Attack Surface Area – entry points in an application that may allow attackers to gain entry and eventually exploit vulnerabilities and compromise the application

The goal is to reduce the number of ways an attacker can exploit the application

Limiting entry points, removing unnecessary features, enforcing principle of least privilege, etc.

Practical Example: A company launches a mobile banking app. Instead of including a full social media feed or complex internal chat system that could have vulnerabilities, they only include core banking features like transfers and balance checks. By disabling unused ports and removing unnecessary third-party plugins from their web server, they further reduce the paths an attacker can take.



2. SECURE DEFAULTS

Assume that users will always stick to the default settings of an application

These settings should be highly secure and enabled by default as much as possible

Security is built-in from the start to reduce the chance of misconfiguration and additional vulnerabilities

MFA, no default credentials, password policies, etc.

Practical Example: When a customer buys a new smart home camera, the device requires them to create a unique password and enable MFA during the initial setup. The camera does not come with a default password like admin or 1234, which prevents unauthorized access from hackers scanning for factory-set credentials.

2. SECURE DEFAULTS

Assume that users will always stick to the default settings of an application

These settings should be highly secure and enabled by default as much as possible

Security is built-in from the start to reduce the chance of misconfiguration and additional vulnerabilities

MFA, no default credentials, password policies, etc.

Practical Example: When a customer buys a new smart home camera, the device requires them to create a unique password and enable MFA during the initial setup. The camera does not come with a default password like admin or 1234, which prevents unauthorized access from hackers scanning for factory-set credentials.

2. SECURE DEFAULTS

Assume that users will always stick to the default settings of an application

These settings should be highly secure and enabled by default as much as possible

Security is built-in from the start to reduce the chance of misconfiguration and additional vulnerabilities

MFA, no default credentials, password policies, etc.

Practical Example: When a customer buys a new smart home camera, the device requires them to create a unique password and enable MFA during the initial setup. The camera does not come with a default password like admin or 1234, which prevents unauthorized access from hackers scanning for factory-set credentials.

2. SECURE DEFAULTS

Assume that users will always stick to the default settings of an application

These settings should be highly secure and enabled by default as much as possible

Security is built-in from the start to reduce the chance of misconfiguration and additional vulnerabilities

MFA, no default credentials, password policies, etc.

Practical Example: When a customer buys a new smart home camera, the device requires them to create a unique password and enable MFA during the initial setup. The camera does not come with a default password like admin or 1234, which prevents unauthorized access from hackers scanning for factory-set credentials.

2. SECURE DEFAULTS

Assume that users will always stick to the default settings of an application

These settings should be highly secure and enabled by default as much as possible

Security is built-in from the start to reduce the chance of misconfiguration and additional vulnerabilities

MFA, no default credentials, password policies, etc.

Practical Example: When a customer buys a new smart home camera, the device requires them to create a unique password and enable MFA during the initial setup. The camera does not come with a default password like admin or 1234, which prevents unauthorized access from hackers scanning for factory-set credentials.

2. SECURE DEFAULTS

Assume that users will always stick to the default settings of an application

These settings should be highly secure and enabled by default as much as possible

Security is built-in from the start to reduce the chance of misconfiguration and additional vulnerabilities

MFA, no default credentials, password policies, etc.

Practical Example: When a customer buys a new smart home camera, the device requires them to create a unique password and enable MFA during the initial setup. The camera does not come with a default password like admin or 1234, which prevents unauthorized access from hackers scanning for factory-set credentials.

3. PRINCIPLE OF LEAST PRIVILEGE

Making sure that users and systems are given only minimum access that they need while interacting with the application

Regular review and audit, precise access control model configurations

Practical Example: In a hospital database, a receptionist is granted access to view patient schedules but cannot view private medical histories or change prescriptions. Only doctors have the privilege to modify medical records, and only the IT administrator can change system configurations. Regular audits ensure that if a receptionist moves to a different department, their old access is revoked.

3. PRINCIPLE OF LEAST PRIVILEGE

Making sure that users and systems are given only minimum access that they need while interacting with the application

Regular review and audit, precise access control model configurations

Practical Example: In a hospital database, a receptionist is granted access to view patient schedules but cannot view private medical histories or change prescriptions. Only doctors have the privilege to modify medical records, and only the IT administrator can change system configurations. Regular audits ensure that if a receptionist moves to a different department, their old access is revoked.

3. PRINCIPLE OF LEAST PRIVILEGE

Making sure that users and systems are given only minimum access that they need while interacting with the application

Regular review and audit, precise access control model configurations

Practical Example: In a hospital database, a receptionist is granted access to view patient schedules but cannot view private medical histories or change prescriptions. Only doctors have the privilege to modify medical records, and only the IT administrator can change system configurations. Regular audits ensure that if a receptionist moves to a different department, their old access is revoked.

3. PRINCIPLE OF LEAST PRIVILEGE

Making sure that users and systems are given only minimum access that they need while interacting with the application

Regular review and audit, precise access control model configurations

Practical Example: In a hospital database, a receptionist is granted access to view patient schedules but cannot view private medical histories or change prescriptions. Only doctors have the privilege to modify medical records, and only the IT administrator can change system configurations. Regular audits ensure that if a receptionist moves to a different department, their old access is revoked.

4. PRINCIPLE OF DEFENSE IN DEPTH

Also known as layered security

Implementing multiple security controls to protect an application

If one layer is breached, there are still other layers to minimize impact and deter the attacker

Network → Host → Application

Practical Example: An office building uses a perimeter fence (Layer 1), a badge-entry system at the front door (Layer 2), security cameras in the halls (Layer 3), and individual locks on server room doors (Layer 4). If a thief jumps the fence, they are still stopped by the locked building door and recorded by the cameras.

4. PRINCIPLE OF DEFENSE IN DEPTH

Also known as layered security

Implementing multiple security controls to protect an application

If one layer is breached, there are still other layers to minimize impact and deter the attacker

Network → Host → Application

Practical Example: An office building uses a perimeter fence (Layer 1), a badge-entry system at the front door (Layer 2), security cameras in the halls (Layer 3), and individual locks on server room doors (Layer 4). If a thief jumps the fence, they are still stopped by the locked building door and recorded by the cameras.

4. PRINCIPLE OF DEFENSE IN DEPTH

Also known as layered security

Implementing multiple security controls to protect an application

If one layer is breached, there are still other layers to minimize impact and deter the attacker

Network → Host → Application

Practical Example: An office building uses a perimeter fence (Layer 1), a badge-entry system at the front door (Layer 2), security cameras in the halls (Layer 3), and individual locks on server room doors (Layer 4). If a thief jumps the fence, they are still stopped by the locked building door and recorded by the cameras.

4. PRINCIPLE OF DEFENSE IN DEPTH

Also known as layered security

Implementing multiple security controls to protect an application

If one layer is breached, there are still other layers to minimize impact and deter the attacker

Network → Host → Application

Practical Example: An office building uses a perimeter fence (Layer 1), a badge-entry system at the front door (Layer 2), security cameras in the halls (Layer 3), and individual locks on server room doors (Layer 4). If a thief jumps the fence, they are still stopped by the locked building door and recorded by the cameras.

4. PRINCIPLE OF DEFENSE IN DEPTH

Also known as layered security

Implementing multiple security controls to protect an application

If one layer is breached, there are still other layers to minimize impact and deter the attacker

Network → Host → Application

Practical Example: An office building uses a perimeter fence (Layer 1), a badge-entry system at the front door (Layer 2), security cameras in the halls (Layer 3), and individual locks on server room doors (Layer 4). If a thief jumps the fence, they are still stopped by the locked building door and recorded by the cameras.

4. PRINCIPLE OF DEFENSE IN DEPTH

Also known as layered security

Implementing multiple security controls to protect an application

If one layer is breached, there are still other layers to minimize impact and deter the attacker

Network → Host → Application

Practical Example: An office building uses a perimeter fence (Layer 1), a badge-entry system at the front door (Layer 2), security cameras in the halls (Layer 3), and individual locks on server room doors (Layer 4). If a thief jumps the fence, they are still stopped by the locked building door and recorded by the cameras.

5. FAIL SECURELY

Assume that your security controls are going to fail at some point

Applications should still respond accordingly in the case of an error or failure to maintain security

Proper error handling, error messages that do not leak sensitive information, disabling debugging mode

Practical Example: When an electronic door lock loses power during a security breach, it is programmed to stay locked ("fail closed") rather than unlocking automatically ("fail open"). In software, if a database connection fails while a user is logging in, the application should show a generic `system error` message rather than a detailed technical log that might reveal database structure to an attacker.

5. FAIL SECURELY

Assume that your security controls are going to fail at some point

Applications should still respond accordingly in the case of an error or failure to maintain security

Proper error handling, error messages that do not leak sensitive information, disabling debugging mode

Practical Example: When an electronic door lock loses power during a security breach, it is programmed to stay locked ("fail closed") rather than unlocking automatically ("fail open"). In software, if a database connection fails while a user is logging in, the application should show a generic `system error` message rather than a detailed technical log that might reveal database structure to an attacker.

5. FAIL SECURELY

Assume that your security controls are going to fail at some point

Applications should still respond accordingly in the case of an error or failure to maintain security

Proper error handling, error messages that do not leak sensitive information, disabling debugging mode

Practical Example: When an electronic door lock loses power during a security breach, it is programmed to stay locked ("fail closed") rather than unlocking automatically ("fail open"). In software, if a database connection fails while a user is logging in, the application should show a generic `system error` message rather than a detailed technical log that might reveal database structure to an attacker.

5. FAIL SECURELY

Assume that your security controls are going to fail at some point

Applications should still respond accordingly in the case of an error or failure to maintain security

Proper error handling, error messages that do not leak sensitive information, disabling debugging mode

Practical Example: When an electronic door lock loses power during a security breach, it is programmed to stay locked ("fail closed") rather than unlocking automatically ("fail open"). In software, if a database connection fails while a user is logging in, the application should show a generic `system error` message rather than a detailed technical log that might reveal database structure to an attacker.

5. FAIL SECURELY

Assume that your security controls are going to fail at some point

Applications should still respond accordingly in the case of an error or failure to maintain security

Proper error handling, error messages that do not leak sensitive information, disabling debugging mode

Practical Example: When an electronic door lock loses power during a security breach, it is programmed to stay locked ("fail closed") rather than unlocking automatically ("fail open"). In software, if a database connection fails while a user is logging in, the application should show a generic `system error` message rather than a detailed technical log that might reveal database structure to an attacker.

6. EXTERNAL SYSTEMS ARE INSECURE

Consider that all external systems are not safe

Third-party services, APIs, user inputs

Always monitor and log the interaction to and from these external dependencies

Do proper research before use such as existing vulnerabilities

Practical Example: An e-commerce site uses a third-party service to process payments. Instead of trusting the service blindly, the site validates all data returned from the payment API and logs every transaction. This prevents an attacker from sending a fake payment successful message from a compromised external server to get free goods.

6. EXTERNAL SYSTEMS ARE INSECURE

Consider that all external systems are not safe

Third-party services, APIs, user inputs

Always monitor and log the interaction to and from these external dependencies

Do proper research before use such as existing vulnerabilities

Practical Example: An e-commerce site uses a third-party service to process payments. Instead of trusting the service blindly, the site validates all data returned from the payment API and logs every transaction. This prevents an attacker from sending a fake payment successful message from a compromised external server to get free goods.

6. EXTERNAL SYSTEMS ARE INSECURE

Consider that all external systems are not safe

Third-party services, APIs, user inputs

Always monitor and log the interaction to and from these external dependencies

Do proper research before use such as existing vulnerabilities

Practical Example: An e-commerce site uses a third-party service to process payments. Instead of trusting the service blindly, the site validates all data returned from the payment API and logs every transaction. This prevents an attacker from sending a fake payment successful message from a compromised external server to get free goods.

6. EXTERNAL SYSTEMS ARE INSECURE

Consider that all external systems are not safe

Third-party services, APIs, user inputs

Always monitor and log the interaction to and from these external dependencies

Do proper research before use such as existing vulnerabilities

Practical Example: An e-commerce site uses a third-party service to process payments. Instead of trusting the service blindly, the site validates all data returned from the payment API and logs every transaction. This prevents an attacker from sending a fake payment successful message from a compromised external server to get free goods.

6. EXTERNAL SYSTEMS ARE INSECURE

Consider that all external systems are not safe

Third-party services, APIs, user inputs

Always monitor and log the interaction to and from these external dependencies

Do proper research before use such as existing vulnerabilities

Practical Example: An e-commerce site uses a third-party service to process payments. Instead of trusting the service blindly, the site validates all data returned from the payment API and logs every transaction. This prevents an attacker from sending a fake payment successful message from a compromised external server to get free goods.

6. EXTERNAL SYSTEMS ARE INSECURE

Consider that all external systems are not safe

Third-party services, APIs, user inputs

Always monitor and log the interaction to and from these external dependencies

Do proper research before use such as existing vulnerabilities

Practical Example: An e-commerce site uses a third-party service to process payments. Instead of trusting the service blindly, the site validates all data returned from the payment API and logs every transaction. This prevents an attacker from sending a fake payment successful message from a compromised external server to get free goods.

7. SEPARATION OF DUTIES

No single role should have too much authority

User with more duties = more susceptible to making poor decisions and committing mistakes

Assign different users to different roles such as sales manager and product manager

Practical Example: In a software company, the developer who writes the code for a financial transaction system is not the same person who authorizes the code to be deployed to the live server. This principle prevents a single rogue employee from writing and secretly deploying malicious code to steal funds.

7. SEPARATION OF DUTIES

No single role should have too much authority

User with more duties = more susceptible to making poor decisions and committing mistakes

Assign different users to different roles such as sales manager and product manager

Practical Example: In a software company, the developer who writes the code for a financial transaction system is not the same person who authorizes the code to be deployed to the live server. This principle prevents a single rogue employee from writing and secretly deploying malicious code to steal funds.

7. SEPARATION OF DUTIES

No single role should have too much authority

User with more duties = more susceptible to making poor decisions and committing mistakes

Assign different users to different roles such as sales manager and product manager

Practical Example: In a software company, the developer who writes the code for a financial transaction system is not the same person who authorizes the code to be deployed to the live server. This principle prevents a single rogue employee from writing and secretly deploying malicious code to steal funds.

7. SEPARATION OF DUTIES

No single role should have too much authority

User with more duties = more susceptible to making poor decisions and committing mistakes

Assign different users to different roles such as sales manager and product manager

Practical Example: In a software company, the developer who writes the code for a financial transaction system is not the same person who authorizes the code to be deployed to the live server. This principle prevents a single rogue employee from writing and secretly deploying malicious code to steal funds.

7. SEPARATION OF DUTIES

No single role should have too much authority

User with more duties = more susceptible to making poor decisions and committing mistakes

Assign different users to different roles such as sales manager and product manager

Practical Example: In a software company, the developer who writes the code for a financial transaction system is not the same person who authorizes the code to be deployed to the live server. This principle prevents a single rogue employee from writing and secretly deploying malicious code to steal funds.

8. SECURITY THROUGH OBSCURITY IS NOT SECURITY

An application's security does not rely entirely on secrets or information not known by many people

Not disclosing details such as system details and security controls used do not really improve the security of the application

Do not use hidden URLs and/or hardcoded credentials in code

Practical Example: A developer thinks their admin panel is safe because they renamed the URL from `/admin` to `/hidden-admin-login-page-99`. However, an attacker using a web crawler or looking at the browser's network history can easily find it. Real security would involve requiring a strong password and MFA regardless of what the URL is named.

8. SECURITY THROUGH OBSCURITY IS NOT SECURITY

An application's security does not rely entirely on secrets or information not known by many people

Not disclosing details such as system details and security controls used do not really improve the security of the application

Do not use hidden URLs and/or hardcoded credentials in code

Practical Example: A developer thinks their admin panel is safe because they renamed the URL from `/admin` to `/hidden-admin-login-page-99`. However, an attacker using a web crawler or looking at the browser's network history can easily find it. Real security would involve requiring a strong password and MFA regardless of what the URL is named.

8. SECURITY THROUGH OBSCURITY IS NOT SECURITY

An application's security does not rely entirely on secrets or information not known by many people

Not disclosing details such as system details and security controls used do not really improve the security of the application

Do not use hidden URLs and/or hardcoded credentials in code

Practical Example: A developer thinks their admin panel is safe because they renamed the URL from `/admin` to `/hidden-admin-login-page-99`. However, an attacker using a web crawler or looking at the browser's network history can easily find it. Real security would involve requiring a strong password and MFA regardless of what the URL is named.

8. SECURITY THROUGH OBSCURITY IS NOT SECURITY

An application's security does not rely entirely on secrets or information not known by many people

Not disclosing details such as system details and security controls used do not really improve the security of the application

Do not use hidden URLs and/or hardcoded credentials in code

Practical Example: A developer thinks their admin panel is safe because they renamed the URL from `/admin` to `/hidden-admin-login-page-99`. However, an attacker using a web crawler or looking at the browser's network history can easily find it. Real security would involve requiring a strong password and MFA regardless of what the URL is named.

8. SECURITY THROUGH OBSCURITY IS NOT SECURITY

An application's security does not rely entirely on secrets or information not known by many people

Not disclosing details such as system details and security controls used do not really improve the security of the application

Do not use hidden URLs and/or hardcoded credentials in code

Practical Example: A developer thinks their admin panel is safe because they renamed the URL from `/admin` to `/hidden-admin-login-page-99`. However, an attacker using a web crawler or looking at the browser's network history can easily find it. Real security would involve requiring a strong password and MFA regardless of what the URL is named.

9. SIMPLICITY

Simpler design is easier to test

Complex systems have more potential vulnerabilities and prone to user misconfiguration

Practical Example: A company chooses to use a standard, well-documented authentication protocol (like OAuth) rather than building their own custom, highly complex login system. Because the standard protocol is simple and widely tested, there is a much lower chance of a developer making a mistake that leads to a security hole.

9. SIMPLICITY

Simpler design is easier to test

Complex systems have more potential vulnerabilities and prone to user misconfiguration

Practical Example: A company chooses to use a standard, well-documented authentication protocol (like OAuth) rather than building their own custom, highly complex login system. Because the standard protocol is simple and widely tested, there is a much lower chance of a developer making a mistake that leads to a security hole.

9. SIMPLICITY

Simpler design is easier to test

Complex systems have more potential vulnerabilities and prone to user misconfiguration

Practical Example: A company chooses to use a standard, well-documented authentication protocol (like OAuth) rather than building their own custom, highly complex login system. Because the standard protocol is simple and widely tested, there is a much lower chance of a developer making a mistake that leads to a security hole.

9. SIMPLICITY

Simpler design is easier to test

Complex systems have more potential vulnerabilities and prone to user misconfiguration

Practical Example: A company chooses to use a standard, well-documented authentication protocol (like OAuth) rather than building their own custom, highly complex login system. Because the standard protocol is simple and widely tested, there is a much lower chance of a developer making a mistake that leads to a security hole.

10. FIX ISSUES CORRECTLY

Understand the root cause of security issues before applying a fix

Apply comprehensive fixes that fully eliminate vulnerabilities, not just mitigate them partially

Test fixes thoroughly to ensure they do not introduce new security issues

Follow security and SDLC (Software Development Life Cycle) best practices and use proven techniques instead of custom or temporary solutions

Practical Example: If a website is vulnerable to SQL injection because it doesn't filter user input, a quick fix might be to block specific keywords like `DROP TABLE`. A correct fix would be to switch to parameterized queries which addresses the root cause by ensuring the database treats all user input as data, not as executable commands.

10. FIX ISSUES CORRECTLY

Understand the root cause of security issues before applying a fix

Apply comprehensive fixes that fully eliminate vulnerabilities, not just mitigate them partially

Test fixes thoroughly to ensure they do not introduce new security issues

Follow security and SDLC (Software Development Life Cycle) best practices and use proven techniques instead of custom or temporary solutions

Practical Example: If a website is vulnerable to SQL injection because it doesn't filter user input, a quick fix might be to block specific keywords like `DROP TABLE`. A correct fix would be to switch to parameterized queries which addresses the root cause by ensuring the database treats all user input as data, not as executable commands.

10. FIX ISSUES CORRECTLY

Understand the root cause of security issues before applying a fix

Apply comprehensive fixes that fully eliminate vulnerabilities, not just mitigate them partially

Test fixes thoroughly to ensure they do not introduce new security issues

Follow security and SDLC (Software Development Life Cycle) best practices and use proven techniques instead of custom or temporary solutions

Practical Example: If a website is vulnerable to SQL injection because it doesn't filter user input, a quick fix might be to block specific keywords like `DROP TABLE`. A correct fix would be to switch to parameterized queries which addresses the root cause by ensuring the database treats all user input as data, not as executable commands.

10. FIX ISSUES CORRECTLY

Understand the root cause of security issues before applying a fix

Apply comprehensive fixes that fully eliminate vulnerabilities, not just mitigate them partially

Test fixes thoroughly to ensure they do not introduce new security issues

Follow security and SDLC (Software Development Life Cycle) best practices and use proven techniques instead of custom or temporary solutions

Practical Example: If a website is vulnerable to SQL injection because it doesn't filter user input, a quick fix might be to block specific keywords like `DROP TABLE`. A correct fix would be to switch to parameterized queries which addresses the root cause by ensuring the database treats all user input as data, not as executable commands.

10. FIX ISSUES CORRECTLY

Understand the root cause of security issues before applying a fix

Apply comprehensive fixes that fully eliminate vulnerabilities, not just mitigate them partially

Test fixes thoroughly to ensure they do not introduce new security issues

Follow security and SDLC (Software Development Life Cycle) best practices and use proven techniques instead of custom or temporary solutions

Practical Example: If a website is vulnerable to SQL injection because it doesn't filter user input, a quick fix might be to block specific keywords like `DROP TABLE`. A correct fix would be to switch to parameterized queries which addresses the root cause by ensuring the database treats all user input as data, not as executable commands.

10. FIX ISSUES CORRECTLY

Understand the root cause of security issues before applying a fix

Apply comprehensive fixes that fully eliminate vulnerabilities, not just mitigate them partially

Test fixes thoroughly to ensure they do not introduce new security issues

Follow security and SDLC (Software Development Life Cycle) best practices and use proven techniques instead of custom or temporary solutions

Practical Example: If a website is vulnerable to SQL injection because it doesn't filter user input, a quick fix might be to block specific keywords like `DROP TABLE`. A correct fix would be to switch to parameterized queries which addresses the root cause by ensuring the database treats all user input as data, not as executable commands.

SECURITY PRINCIPLES FOR SECURE SOFTWARE DESIGN

Secure Web Development